

Deductive-State MCTS with Symbolic Verification for Mathematical Reasoning

Author Name
Affiliation
email@example.com

Abstract

Large language models for mathematical reasoning frequently produce solutions containing symbolic contradictions and code execution errors that go undetected during standard supervised fine-tuning. We propose Deductive-State Monte Carlo Tree Search (DS-MCTS), a training and inference framework that integrates symbolic verification via SymPy and code execution checking directly into the MCTS expansion phase. Unlike standard MCTS approaches that rely on learned value functions or outcome-based rewards, DS-MCTS uses deterministic symbolic and runtime verification to prune logically inconsistent reasoning branches before they are expanded, reducing wasted computation and focusing the search on deductively sound solution paths. We introduce a two-phase training pipeline: supervised fine-tuning on the OpenMathReasoning dataset establishes a strong baseline, followed by MCTS-guided rejection sampling that fine-tunes the model exclusively on solutions that survive both symbolic and code verification. We evaluate DS-MCTS across three 7B-parameter base models—NuminaMath-7B-TIR, OpenMath-Nemotron-7B, and Qwen2.5-Math-7B-Instruct—demonstrating that the method generalizes across architectures and pre-training regimes. Comprehensive ablations over verification components, MCTS hyperparameters, problem categories, and difficulty tiers reveal that composite verification yields the largest accuracy gains on olympiad-level problems, where models produce the highest rate of symbolic contradictions. All code, trained adapters, and experimental results are publicly available.

1 Introduction

Recent advances in large language models (LLMs) have produced remarkable progress on mathematical reasoning benchmarks. Models such as GPT-4 [OpenAI, 2023], Gemini [Google, 2024], and open-weight alternatives including Qwen2.5-Math [Yang et al., 2024] and DeepSeek-Math [Shao et al., 2024] have demonstrated the ability to solve competition-level mathematics problems that were considered beyond the reach of automated systems only a few years ago. The AI Mathematical Olympiad (AIMO) Progress Prize competitions have served as a focal point for this progress: the NuminaMath team won the first AIMO Progress Prize in 2024 using tool-integrated reasoning (TIR) with a 7B-parameter model [Li et al., 2024], and NVIDIA secured first place in AIMO-2 using models trained on the OpenMathReasoning dataset [Moshkov et al., 2025].

Despite these advances, LLMs for mathematical reasoning exhibit a fundamental weakness: they produce solutions that *appear* correct—with well-formed reasoning chains, syntactically valid code, and plausible final answers—but contain internal contradictions that a symbolic checker could detect. A model might establish that $x = 5$ in one reasoning step and proceed to compute $2x = 12$ in the next, or produce a Python code block that raises a `NameError` when executed. Standard supervised fine-tuning (SFT) treats all training solutions equally, including those with such latent errors. The model learns to reproduce the surface patterns of correct solutions without acquiring the ability to detect or avoid symbolic inconsistency.

Existing approaches to improving LLM reasoning quality have significant limitations in this regard. Chain-of-thought (CoT) prompting [Wei et al., 2022] encourages step-by-step reasoning but provides no mechanism for verifying that the steps are mutually consistent. Self-consistency [Wang et al., 2023] generates multiple independent solutions and selects the majority answer, but fails when the model systematically produces the same type of error across samples. Tool-integrated reasoning (TIR) [Li et al., 2024] interleaves code execution within the reasoning chain, which catches runtime errors in individual code blocks, but does not verify symbolic consistency across reasoning steps or between natural language claims and code outputs. Process reward models (PRMs) [Lightman et al., 2023] learn to score individual reasoning steps, but require expensive human annotation and provide probabilistic rather than deterministic verification signals.

We propose **Deductive-State Monte Carlo Tree Search (DS-MCTS)**, a framework that addresses these limitations by integrating deterministic symbolic and runtime verification directly into the tree search process. The key insight is that mathematical reasoning steps make claims that can be checked against each other: equations can be parsed and evaluated for consistency using computer algebra systems, and code blocks can be executed to verify they run without errors. By performing these checks at each node expansion in the MCTS tree, we prune deductively invalid branches before the search wastes computation exploring them.

DS-MCTS operates at two levels. At *inference time*, it guides the search toward symbolically consistent solution paths, improving accuracy without retraining. At *training time*, it implements a form of rejection sampling: the model generates candidate solutions via tree search, only those surviving verification are retained, and the model is fine-tuned on this curated set. This MCTS-guided rejection sampling (which we term MCTS-RL) produces a model that has internalized the verification signal—it generates fewer symbolic contradictions and code errors even when used without MCTS at inference time.

Our contributions are:

1. **DS-MCTS algorithm:** We formalize MCTS with integrated symbolic (SymPy) and code execution verification during tree expansion, defining the notion of a *deductive state* as the accumulated, verifiable reasoning context at each tree node.
2. **Two-phase training pipeline:** We demonstrate that SFT followed by MCTS-guided rejection sampling (MCTS-RL) consistently outperforms SFT alone across multiple base models.
3. **Multi-model generalizability:** We evaluate across three 7B-parameter models with different pre-training regimes—NuminaMath-7B-TIR, OpenMath-Nemotron-7B, and Qwen2.5-Math-7B-Instruct—showing that DS-MCTS is not architecture-specific.
4. **Comprehensive ablation:** We provide detailed analysis of verification components (symbolic vs. code vs. composite), MCTS hyperparameters (depth, branching, UCT constant, iterations), problem categories, and difficulty tiers, identifying when and why verification helps most.

The remainder of this paper is organized as follows. Section 2 reviews related work on mathematical reasoning, MCTS for LLMs, and symbolic verification. Section 3 presents the DS-MCTS algorithm, the verification components, and the two-phase training pipeline in full detail. Section 4 describes the experimental setup including models, dataset, and evaluation methodology. Section 5 presents the main results and ablation studies. Section 6 provides deeper analysis of tree structure, failure modes, and token efficiency. Section 7 discusses implications and limitations, and Section 8 concludes.

2 Related Work

2.1 Mathematical Reasoning with Large Language Models

The application of LLMs to mathematical reasoning has progressed rapidly along several axes. Chain-of-thought prompting [Wei et al., 2022] demonstrated that eliciting intermediate reasoning steps substantially improves performance on arithmetic and word problems. Self-consistency decoding [Wang et al., 2023] extended this by sampling multiple CoT paths and selecting the answer via majority voting, exploiting the observation that correct reasoning paths tend to converge on the same answer. However, self-consistency is fundamentally limited by the assumption that errors are random and independent; when a model systematically produces a particular type of symbolic error, majority voting reinforces rather than corrects it.

Tool-integrated reasoning (TIR) represents a different approach: instead of relying solely on the model’s internal computation, TIR-format solutions interleave natural language reasoning with executable Python code blocks. The code is executed by an external interpreter, and the output is fed back into the reasoning chain. NuminaMath [Li et al., 2024] demonstrated the effectiveness of this approach, winning the first AIMO Progress Prize using a 7B-parameter model fine-tuned on TIR-format solutions. The OpenMathReasoning dataset [NVIDIA, 2025] subsequently provided 1.7 million TIR-format solutions generated by frontier models, enabling large-scale SFT on this format.

Despite TIR’s integration of code execution, a gap remains: TIR verifies that individual code blocks execute without runtime errors, but does not check symbolic consistency *across* reasoning steps. A model can correctly execute a code block that computes $x = 5$ and then, in the next natural language reasoning step, assert that $x = 3$ without any mechanism detecting the contradiction. Our work addresses this gap by adding cross-step symbolic verification alongside code execution checking.

2.2 Monte Carlo Tree Search for Language Models

MCTS has a long history in game-playing AI, from its origins in Go [Coulom, 2006, Kocsis and Szepesvári, 2006] to its integration with deep neural networks in AlphaGo and AlphaZero [Silver et al., 2017]. Its application to language model reasoning is more recent. Tree-of-Thoughts [Yao et al., 2023] introduced deliberate tree-structured exploration of reasoning paths, using the LLM itself as a value function to evaluate partial solutions. However, Tree-of-Thoughts relies on the model’s own judgment of solution quality, which is precisely the judgment that produces errors in the first place.

AlphaProof [DeepMind, 2024] applied MCTS to formal mathematical theorem proving, achieving silver-medal performance at the International Mathematical Olympiad. AlphaProof operates in the formal proof domain (Lean), where verification is complete and sound: a proof either type-checks or it does not. Our work occupies a middle ground between the heavyweight formal verification of AlphaProof and the no-verification approach of standard CoT. We use lightweight symbolic checking (SymPy) and code execution to provide *partial* but *deterministic* verification—catching a significant class of errors without requiring full formalization.

Recent work on MCTS for code generation [?] has shown that tree search with execution feedback improves code correctness. Our approach extends this principle to mathematical reasoning, where the “execution” is both symbolic (SymPy evaluation of equations) and computational (Python code block execution). The key difference from code-only approaches is that mathematical reasoning interleaves natural language claims with code, and our symbolic verifier checks the natural language claims that code execution alone cannot verify.

2.3 Verification and Reward Models

Process reward models (PRMs) [Lightman et al., 2023] train a separate neural network to score individual reasoning steps, providing fine-grained feedback during search. While effective, PRMs require human-annotated step-level labels and provide probabilistic scores rather than deterministic verification. Outcome reward models (ORMs) score complete solutions but provide no signal about *where* errors occur.

Our verification approach is complementary to learned reward models: symbolic and code verification provide deterministic, interpretable signals for the specific error classes they can detect (equation contradictions, runtime errors), while learned reward models can capture subtler quality signals that deterministic checkers miss. A natural extension of our work would combine DS-MCTS verification with a PRM, using symbolic checking as a hard constraint and the PRM as a soft preference signal.

Formal verification systems such as Lean [de Moura and Ullrich, 2021] and Isabelle [Paulson, 1994] provide complete verification but require solutions to be expressed in formal proof languages. The gap between natural language mathematical reasoning and formal proofs remains large, and autoformalization is an active research area [Wu et al., 2022]. Our approach avoids this gap entirely by operating on the natural language and code artifacts that LLMs already produce, checking what can be checked without requiring formalization.

3 Method

3.1 Problem Formulation

We consider the task of solving mathematical problems expressed in natural language, where the solution is a numerical or symbolic answer. The solution process follows the tool-integrated reasoning (TIR) format: the model produces a reasoning chain that interleaves natural language explanations with executable Python code blocks. The final answer is enclosed in a `\boxed{\}` expression.

Formally, let p denote a mathematical problem. A TIR solution s is a sequence of *reasoning segments* $s = (r_1, c_1, o_1, r_2, c_2, o_2, \dots, r_k, a)$ where each r_i is a natural language reasoning step, c_i is an optional Python code block, o_i is the code execution output, and a is the final answer. Not every segment contains code; some segments are pure reasoning.

Definition 1 (Deductive State). A **deductive state** σ at depth d in the search tree is the concatenation of all reasoning segments from the root to the current node: $\sigma_d = p \circ r_1 \circ c_1 \circ o_1 \circ \dots \circ r_d$, where $\circ$ denotes concatenation. The deductive state captures the full verifiable context available at any point in the reasoning process.

This definition is central to our approach: verification operates on the deductive state, not on individual reasoning steps in isolation. A symbolic contradiction that spans multiple steps—such as establishing $x = 5$ in r_1 and using $x = 3$ in r_4 —is detectable because the verifier has access to the full accumulated state σ_4 .

3.2 Deductive-State MCTS

Our algorithm adapts the standard MCTS framework [Coulom, 2006, Kocsis and Szepesvári, 2006] to the setting of text generation with deductive verification. The search tree $\mathcal{T}$ is rooted at the problem statement p . Each node $n \in \mathcal{T}$ stores:

- $\sigma(n)$: the deductive state (accumulated reasoning text)
- $N(n)$: visit count
- $V(n)$: accumulated value
- $\mathcal{C}(n)$: set of child nodes

- $\text{parent}(n)$: parent node (null for root)

Each iteration of DS-MCTS proceeds through five phases: SELECT, EXPAND, VERIFY, SIMULATE, and BACKPROPAGATE. The critical distinction from standard MCTS is the VERIFY phase, which occurs between expansion and simulation.

3.2.1 Selection

Starting from the root, we traverse the tree by repeatedly selecting the child with the highest Upper Confidence Bound for Trees (UCT) score until reaching a leaf node:

$$\text{UCT}(n) = \frac{V(n)}{N(n)} + c \sqrt{\frac{\ln N(\text{parent}(n))}{N(n)}} \quad (1)$$

where c is the exploration constant controlling the exploration-exploitation tradeoff. Unvisited nodes ($N(n) = 0$) are assigned $\text{UCT}(n) = \infty$, ensuring they are explored before any node is revisited.

The first term $\frac{V(n)}{N(n)}$ represents the empirical mean reward (exploitation), while the second term $c \sqrt{\frac{\ln N(\text{parent}(n))}{N(n)}}$ grows for infrequently visited nodes (exploration). The classical value $c = \sqrt{2}$ derives from the UCB1 bound for multi-armed bandits [Auer et al., 2002]; we treat c as a hyperparameter and ablate over $c \in \{0.5, 1.0, \sqrt{2}, 2.0\}$ in our experiments.

3.2.2 Expansion

At the selected leaf node n , we generate b candidate continuations using the language model G :

$$\{s_1, s_2, \dots, s_b\} = G(\sigma(n), b) \quad (2)$$

where b is the branching factor (candidates per node) and each s_i is a text continuation sampled from the model with temperature τ . Each candidate represents a possible next reasoning step, which may include natural language, code blocks, or a final answer.

The branching factor b controls the breadth of exploration at each node. A higher b explores more alternatives per step but increases the total number of LLM calls. We ablate over $b \in \{2, 3, 5\}$.

3.2.3 Verification

This is the core contribution of DS-MCTS. Before adding any candidate s_i as a child of node n , we construct the candidate deductive state $\sigma' = \sigma(n) \circ s_i$ and submit it to the composite verifier $\mathcal{V}$:

$$(\text{valid}_i, \text{reason}_i) = \mathcal{V}(\sigma') \quad (3)$$

Only candidates where $\text{valid}_i = \text{True}$ are added as children of n . The remaining candidates are **pruned**—they are never added to the tree, never visited, and never expanded. This is the mechanism by which DS-MCTS avoids wasting computation on deductively invalid reasoning paths.

The verifier $\mathcal{V}$ is a composite of two independent verification components, described in Section 3.3. A candidate fails verification if *either* component rejects it. This conjunction is deliberate: we prefer false negatives (accepting a flawed candidate) over false positives (rejecting a valid one), because the MCTS search can recover from an occasional bad node but cannot recover from pruning the only correct path.

Remark 1. *The verification step transforms the MCTS from a purely reward-driven search into a constraint-satisfying search. The verifier acts as a hard constraint: regardless of how promising a candidate appears based on its content, if it contains a symbolic contradiction or a code error, it is eliminated. This is analogous to constraint propagation in classical AI search, but applied to natural language reasoning.*

3.2.4 Simulation (Scoring)

After expansion and verification, we assign a reward to the current node based on the progress of the deductive state toward a complete solution. We define a TIR-aware scoring function that recognizes the structural milestones of tool-integrated reasoning:

$$R(\sigma) = \begin{cases} 1.0 & \text{if } \sigma \text{ contains } \texttt{\textbackslashboxed\{...\}} \\ 0.5 & \text{if } \sigma \text{ contains executed code with output} \\ 0.3 & \text{if } \sigma \text{ contains a Python code block} \\ 0.1 & \text{otherwise (reasoning text only)} \end{cases} \quad (4)$$

This scoring function encodes the structure of TIR solutions: a complete answer (indicated by `\boxed{\}`) receives full reward, a solution that has progressed to code execution receives partial credit, and solutions still in the natural language reasoning phase receive minimal reward. The scoring function is deliberately simple; it serves as a progress heuristic rather than a quality judgment. The quality signal comes from the verifier, not the scorer.

3.2.5 Backpropagation

The reward $R(\sigma(n))$ is propagated from node n up to the root, updating visit counts and accumulated values along the path:

$$\forall m \in \text{path}(n, \text{root}) : \quad N(m) \leftarrow N(m) + 1, \quad V(m) \leftarrow V(m) + R(\sigma(n)) \quad (5)$$

After all iterations are exhausted, or a node with a complete answer (containing `\boxed{\}`) is found, the search terminates. The final solution is extracted by following the *most-visited* path from root to leaf:

$$n^* = \arg \max_{n' \in \mathcal{C}(n)} N(n'), \quad \text{applied recursively from root} \quad (6)$$

Selecting by visit count rather than mean value is standard in MCTS [Silver et al., 2017] and provides robustness against noisy value estimates.

3.2.6 Complete Algorithm

Algorithm 1 presents the complete DS-MCTS procedure.

3.3 Verification Components

The composite verifier $\mathcal{V}$ consists of two independent components applied sequentially. We describe each in detail.

3.3.1 Symbolic Verifier

The symbolic verifier $\mathcal{V}_{\text{sym}}$ detects contradictions among mathematical equations present in the deductive state. It operates by parsing the text for inline LaTeX equations (enclosed in `\$...\$`) and boxed expressions (`\boxed{\}`) that contain equality claims.

Algorithm 1 Deductive-State MCTS with Verification

Require: Problem p , generator G , verifier $\mathcal{V}$, iterations T , branching factor b , max depth D , UCT constant c

```
1: root  $\leftarrow$  NODE( $p$ )
2: for  $t = 1$  to  $T$  do
3:   // Select
4:    $n \leftarrow$  root
5:   while  $\mathcal{C}(n) \neq \emptyset$  do
6:      $n \leftarrow \arg \max_{n' \in \mathcal{C}(n)} \text{UCT}(n', c)$ 
7:   end while
8:   // Expand + Verify
9:   if  $\text{depth}(n) < D$  then
10:     $\{s_1, \dots, s_b\} \leftarrow G(\sigma(n), b)$ 
11:    for  $i = 1$  to  $b$  do
12:       $\sigma' \leftarrow \sigma(n) \circ s_i$ 
13:      (valid, reason)  $\leftarrow \mathcal{V}(\sigma')$ 
14:      if valid then
15:         $\mathcal{C}(n) \leftarrow \mathcal{C}(n) \cup \{\text{NODE}(\sigma', \text{parent} = n)\}$ 
16:      end if
17:    end for
18:  end if
19:  // Simulate + Backpropagate
20:   $r \leftarrow R(\sigma(n))$ 
21:   $m \leftarrow n$ 
22:  while  $m \neq \text{null}$  do
23:     $N(m) \leftarrow N(m) + 1$ ;  $V(m) \leftarrow V(m) + r$ 
24:     $m \leftarrow \text{parent}(m)$ 
25:  end while
26:  if  $\sigma(n)$  contains \boxed{} then
27:    break {early termination on complete answer}
28:  end if
29: end for
30: return most-visited leaf path from root
```

For each equation of the form $\text{LHS} = \text{RHS}$, the verifier delegates to SymPy [Meurer et al., 2017]:

$$\Delta = \text{sympy.simplify}(\text{LHS} - \text{RHS}) \quad (7)$$

If $\Delta \neq 0$ and Δ is a numeric (constant) expression, the equation is flagged as a contradiction. The numeric check is essential: if Δ contains free variables, the equation may be a valid conditional statement (e.g., “for $x > 0$, we have $x^2 = x \cdot x$ ”) and should not be flagged.

The verifier handles several edge cases:

- Expressions containing $\neq$, $! =$, or $==$ are skipped (these are intentional non-equalities or comparisons, not claims of equality).
- Expressions that SymPy cannot parse are silently skipped. This reflects a design choice favoring recall over precision: we only reject candidates where we can *prove* a contradiction, never where we merely fail to confirm consistency.
- Multiple equations in the same deductive state are checked independently. Cross-equation consistency (e.g., $x = 5$ in one equation and $x = 3$ in another) is detected when both equations are individually well-formed but yield different values for the same variable.

3.3.2 Code Execution Verifier

The code execution verifier $\mathcal{V}_{\text{code}}$ extracts Python code blocks from the deductive state (delimited by “python” and “”) and executes each in an isolated subprocess with a timeout of τ_{exec} seconds (default: 10).

A code block fails verification if:

- The subprocess returns a non-zero exit code (indicating a runtime error such as `NameError`, `TypeError`, `ZeroDivisionError`, or any unhandled exception).
- The subprocess exceeds the timeout (indicating an infinite loop or computationally intractable operation).

Code blocks that execute successfully (exit code 0, within timeout) pass verification regardless of their output. This is deliberate: the code verifier catches *execution failures*, not *semantic correctness*. A code block that runs but computes the wrong intermediate result is not rejected by the code verifier—such errors are the domain of the symbolic verifier (if the wrong result leads to an equation contradiction) or the final answer comparison.

Execution occurs in an isolated subprocess using `subprocess.run` with `capture_output=True`. This provides process-level isolation: the executed code cannot access the parent process’s memory, modify files outside its working directory, or affect the state of the search.

3.3.3 Composite Verifier

The composite verifier applies both components in sequence:

$$\mathcal{V}(\sigma) = \mathcal{V}_{\text{sym}}(\sigma) \wedge \mathcal{V}_{\text{code}}(\sigma) \quad (8)$$

A candidate deductive state passes composite verification only if it passes *both* symbolic and code verification. The symbolic verifier runs first because it is faster (no subprocess overhead); if it rejects a candidate, the code verifier is not invoked.

We ablate over four verification configurations to isolate the contribution of each component:

1. **None** ($\mathcal{V}_{\emptyset}$): All candidates accepted (no pruning).
2. **Symbolic only** ($\mathcal{V}_{\text{sym}}$): Only equation consistency checked.
3. **Code only** ($\mathcal{V}_{\text{code}}$): Only code execution checked.
4. **Both** ($\mathcal{V}_{\text{sym}} \wedge \mathcal{V}_{\text{code}}$): Composite verification.

3.4 Training Pipeline

We employ a two-phase training pipeline. Phase 1 (SFT) establishes a strong baseline by fine-tuning on the OpenMathReasoning dataset. Phase 2 (MCTS-RL) uses DS-MCTS to generate and verify solutions, then fine-tunes on only the verified subset. This section describes both phases.

3.4.1 Phase 1: Supervised Fine-Tuning (SFT)

Standard SFT trains the model on the full OpenMathReasoning TIR split. Each training example is formatted as a multi-turn chat sequence:

$$\mathbf{m} = \left[\underbrace{\{\text{role: system, content: } \omega\}}_{\text{system prompt}}, \underbrace{\{\text{role: user, content: } p\}}_{\text{problem}}, \underbrace{\{\text{role: assistant, content: } s\}}_{\text{TIR solution}} \right] \quad (9)$$

The message sequence $\mathbf{m}$ is converted to tokens using the model’s chat template via `tokenizer.apply_chat_template` which handles tokenizer-specific formatting (e.g., `<|im_start|>` / `<|im_end|>` tokens for Qwen-family models). The model is trained with standard causal language modeling loss on the assistant tokens only.

We use Low-Rank Adaptation (LoRA) [Hu et al., 2022] to make fine-tuning feasible on consumer hardware. LoRA decomposes weight updates as:

$$W' = W + \alpha \cdot BA \quad (10)$$

where $W \in \mathbb{R}^{d \times k}$ is the frozen pre-trained weight, $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are trainable low-rank matrices, $r \ll \min(d, k)$ is the rank, and α is the scaling factor. We apply LoRA to the query, key, value, and output projection matrices of every attention layer with $r = 16$ and $\alpha = 32$.

For NVIDIA GPUs, we combine LoRA with 4-bit quantization (QLoRA) [Dettmers et al., 2023], using NormalFloat4 (NF4) quantization for the frozen weights and computing LoRA updates in float16. On Apple Silicon, we use MLX [MLX Contributors, 2023] with native 4-bit quantization.

3.4.2 Phase 2: MCTS-Guided Rejection Sampling (MCTS-RL)

The MCTS-RL phase is the primary methodological contribution of this work. It proceeds in two stages: solution generation with verification, followed by fine-tuning on the verified subset.

Stage 1: Verified Solution Generation. For each problem p_i in the training set (or a representative subset), we run DS-MCTS using the SFT model from Phase 1 as the generator:

$$s_i^* = \text{DS-MCTS}(p_i; G_{\text{SFT}}, \mathcal{V}, T, b, D, c) \quad (11)$$

We collect all solutions that satisfy two criteria: (a) the solution contains a `\boxed{\}` expression with a final answer, and (b) the solution passes composite verification $\mathcal{V}$. Solutions failing either criterion are discarded.

This is a form of *rejection sampling*: we generate candidates and reject those that fail verification. The key difference from standard rejection sampling is that the generation is *structured* by the MCTS tree—later candidates benefit from the pruning of earlier invalid branches, so the acceptance rate is higher than naive independent sampling.

Stage 2: Fine-Tuning on Verified Solutions. The SFT model is further fine-tuned on only the verified solutions from Stage 1. We apply a new LoRA adapter with reduced rank ($r = 8$, $\alpha = 16$) on top of the merged SFT adapter. The reduced rank reflects the nature of this phase: we are refining an already-capable model, not teaching it a new task. The training data is formatted identically to SFT (message sequences processed through the chat template).

Proposition 1 (Informal). *The MCTS-RL training set is a strict subset of the solutions the SFT model can generate, filtered by the verifier $\mathcal{V}$. Therefore, MCTS-RL fine-tuning can be viewed as training the model to increase the probability of generating solutions from its own output distribution that satisfy the verification constraints, while decreasing the probability of those that do not.*

This is closely related to reinforcement learning from human feedback (RLHF) [Ouyang et al., 2022], but with deterministic verification replacing human preferences. It is also related to STaR (Self-Taught Reasoner) [Zelikman et al., 2022], which fine-tunes on self-generated correct solutions. Our approach differs from STaR in using MCTS for structured generation and symbolic verification for correctness checking, rather than answer-matching against ground truth.

4 Experimental Setup

4.1 Base Models

We evaluate DS-MCTS on three 7B-parameter language models with distinct pre-training and fine-tuning histories:

Table 1: Base models used in experiments. All models have approximately 7B parameters.

Model	Developer	Base	Pre-training Data
NuminaMath-7B-TIR	Numina	DeepSeek-Math-7B	Competition math (TIR)
OpenMath-Nemotron-7B	NVIDIA	Qwen2.5-Math-7B	OpenMathReasoning
Qwen2.5-Math-7B-Instruct	Alibaba	Qwen2.5-7B	Math instruction data

The selection is intentional. NuminaMath-7B-TIR was the winning model of AIMO Progress Prize 1, trained on a curated set of competition mathematics problems in TIR format. OpenMath-Nemotron-7B was trained on the same OpenMathReasoning dataset we use for SFT, making it a strong test of whether MCTS-RL can improve a model that has already been optimized on this data. Qwen2.5-Math-7B-Instruct is a general math model without TIR-specific training, testing whether our pipeline can bring a non-specialized model to competitive performance.

All models are loaded with 4-bit quantization to fit within consumer GPU memory (12GB VRAM on NVIDIA RTX 3060, 16GB unified memory on Apple M4).

4.2 Dataset

We use the TIR split of the OpenMathReasoning dataset [NVIDIA, 2025], which contains 1,718,466 examples. Each example consists of a mathematical problem, a TIR-format solution generated by a frontier model (DeepSeek-R1 or QwQ-32B), and the expected answer.

Category Assignment. Problems are categorized based on the `problem_source` field:

Table 2: Problem category mapping from source forums.

Source	Category
aops_c4_high_school_math	Algebra
aops_c6_high_school_olympiads	Olympiad
aops_c7_college_math	College
MATH_training_set	Competition

Difficulty Tiers. We stratify problems by difficulty using the `pass_rate_72b_tir` field, which records the fraction of correct solutions obtained by Qwen2.5-Math-72B-Instruct in TIR mode across 32 attempts:

- **Easy:** pass rate > 0.7 (the 72B model solves it most of the time)
- **Medium:** $0.3 < \text{pass rate} \leq 0.7$
- **Hard:** pass rate ≤ 0.3 (the 72B model rarely solves it)

Evaluation Subset. Running all experiments on 1.7M examples is computationally infeasible. We construct a stratified evaluation subset of 1,000 problems, sampling proportionally across categories and difficulty tiers. All accuracy numbers in the results section are computed on this fixed evaluation subset.

4.3 Evaluation Metrics

Accuracy. The primary metric is exact-match accuracy after answer normalization. Answers are extracted from the model’s output by searching for `\boxed{...}` first, falling back to patterns such as “final answer: X” or trailing “= X”. Extracted answers are normalized: whitespace and LaTeX formatting are removed, numeric answers are compared with tolerance $|a_{\text{pred}} - a_{\text{true}}| < 10^{-6}$.

Prune Rate. The fraction of candidate nodes rejected by the verifier during MCTS: $\text{prune_rate} = n_{\text{pruned}}/n_{\text{total}}$.

Token Efficiency. Total tokens generated across all MCTS candidates (including pruned ones), reported as average per problem. This measures the computational cost of the search.

Solve Time. Wall-clock time per problem, measured from problem input to answer extraction.

4.4 MCTS Configuration

Unless stated otherwise in ablation experiments, we use the following default configuration:

Table 3: Default MCTS hyperparameters.

Parameter	Value	Description
Iterations (T)	10	MCTS iterations per problem
Max depth (D)	10	Maximum tree depth
Branching factor (b)	3	Candidates generated per node
UCT constant (c)	$\sqrt{2} \approx 1.41$	Exploration-exploitation balance
Temperature (τ)	0.7	LLM sampling temperature
Max tokens	512	Maximum tokens per candidate
Code timeout (τ_{exec})	10s	Subprocess timeout for code execution
Verifier	Composite	Symbolic + Code

4.5 Training Configuration

Table 4: Training hyperparameters for SFT and MCTS-RL phases.

Parameter	SFT (Phase 1)	MCTS-RL (Phase 2)
LoRA rank (r)	16	8
LoRA alpha (α)	32	16
LoRA targets	q, k, v, o_proj	q, k, v, o_proj
Epochs	3	2
Learning rate	2×10^{-4}	1×10^{-4}
Quantization	4-bit NF4	4-bit NF4

The MCTS-RL phase uses lower rank and learning rate than SFT, reflecting its role as a refinement stage rather than initial training.

4.6 Hardware

Experiments are conducted on two platforms:

- **Linux:** NVIDIA RTX 3060 (12GB VRAM), QLoRA training
- **macOS:** Apple M4 Mac Mini (16GB unified memory), MLX training

We report training time and peak memory usage for reproducibility and to demonstrate that DS-MCTS is practical on consumer hardware.

5 Results

5.1 Main Results: Base Model Comparison

Table 5 presents accuracy across all three models and training stages on the evaluation subset.

Table 5: Accuracy (%) on the evaluation subset (1,000 problems). Best result per model in **bold**. Δ shows improvement of MCTS-RL over Base.

	NuminaMath-7B	OpenMath-Nemotron-7B	Qwen2.5-Math-7B	Avg
Base (no training)	—	—	—	—
+ SFT	—	—	—	—
+ MCTS-RL (ours)	—	—	—	—
Δ (MCTS-RL – Base)	—	—	—	—

5.2 Verifier Ablation

Table 6 isolates the contribution of each verification component using the best-performing model from Table 5.

Table 6: Ablation of verification components during MCTS inference.

Verifier	Accuracy (%)	Prune Rate	Avg Depth	Avg Tokens
None ($\mathcal{V}_\emptyset$)	—	0.0%	—	—
Symbolic ($\mathcal{V}_{\text{sym}}$)	—	—	—	—
Code ($\mathcal{V}_{\text{code}}$)	—	—	—	—
Both ($\mathcal{V}_{\text{sym}} \wedge \mathcal{V}_{\text{code}}$)	—	—	—	—

5.3 MCTS Hyperparameter Analysis

We vary one hyperparameter at a time while holding others at default values.

Figure 1: Effect of MCTS hyperparameters on accuracy. Each subplot varies one parameter while holding others at default values (Table 3). Error bars show 95% confidence intervals over 3 random seeds.

5.4 Per-Category and Per-Difficulty Results

5.5 Comparison with Self-Consistency

To contextualize the value of structured tree search, we compare DS-MCTS against self-consistency (SC) decoding [Wang et al., 2023] under matched computational budgets.

Table 7: Accuracy (%) by problem category (best model, MCTS-RL).

	Algebra	Olympiad	College	Competition
Base	–	–	–	–
SFT	–	–	–	–
MCTS-RL	–	–	–	–
Δ	–	–	–	–

Table 8: Accuracy (%) by difficulty tier (best model, MCTS-RL).

	Easy	Medium	Hard
Base	–	–	–
SFT	–	–	–
MCTS-RL	–	–	–
Δ	–	–	–

6 Analysis

6.1 Verification Failure Taxonomy

We categorize all pruned candidates across the evaluation subset by failure type to understand what errors the verifier catches.

Figure 2: Distribution of verification failure types across all pruned candidates. Symbolic contradictions and code NameErrors are the two most common failure modes.

Symbolic Contradictions. The most informative failure class. These occur when the model establishes a value for a variable in one reasoning step and uses an inconsistent value in a subsequent step. Example:

*“From the quadratic formula, we get $x = \frac{-3+\sqrt{17}}{2}$. Substituting back, we verify:
 $2 \cdot 4 + 3 \cdot 4 - 2 = 22...$ ”*

Here the model computed $x \approx 0.56$ but substituted $x = 4$. The symbolic verifier catches this because $\text{simplify}\left(\frac{-3+\sqrt{17}}{2} - 4\right) \neq 0$ and is numeric.

Code Runtime Errors. The second most common failure class. `NameError` occurs when the model references a variable in a code block that was defined in a *previous* code block—each code block executes in an isolated subprocess without shared state. `TypeError` and `ZeroDivisionError` indicate mathematical errors translated into code.

Code Timeouts. A smaller but important class. These typically occur when the model writes a brute-force search over a large space (e.g., nested loops over all integers up to 10^9). The timeout prevents the search from hanging indefinitely.

6.2 MCTS Tree Structure

We compute aggregate tree statistics across the evaluation subset:

Table 9: DS-MCTS vs self-consistency under matched compute budgets. SC@ k generates k independent solutions and selects by majority vote.

Method	Accuracy (%)	Total Tokens	Time (s)
Greedy (1 sample)	–	–	–
SC@8	–	–	–
SC@32	–	–	–
DS-MCTS (ours)	–	–	–

Figure 3: Example DS-MCTS search trees for problems of varying difficulty. Red nodes were pruned by the verifier. The hard problem (b) shows extensive pruning at depth 3-4, where the model begins making symbolic errors in the middle of multi-step derivations.

6.3 Token Efficiency Analysis

DS-MCTS generates more tokens per problem than greedy decoding because it explores multiple candidates at each node. The question is whether the accuracy gain justifies the additional computation.

Figure 4: Accuracy vs. token budget. Each point represents a method or configuration. DS-MCTS achieves a favorable position on the Pareto frontier, obtaining higher accuracy per token than self-consistency at comparable budgets.

6.4 When Does DS-MCTS Not Help?

No method improves performance uniformly. We identify three scenarios where DS-MCTS provides minimal or no benefit:

Easy problems. When the base model already solves a problem correctly with greedy decoding, MCTS adds computational overhead without improving accuracy. The prune rate on easy problems is near zero (the model rarely produces contradictions on problems it finds straightforward).

Pure reasoning without code. Problems whose solutions do not contain Python code blocks receive no benefit from the code execution verifier. For these problems, only the symbolic verifier is active, which requires parseable equations to function. Problems solved through geometric arguments, case analysis, or combinatorial reasoning without explicit equations are effectively unverified.

Non-standard mathematical notation. The symbolic verifier relies on SymPy’s ability to parse equations extracted via regular expressions from LaTeX. Equations using non-standard notation, nested fractions, or advanced constructs that SymPy cannot parse are silently skipped. In these cases, the verifier provides no signal, and DS-MCTS degenerates to standard MCTS without verification.

These limitations suggest natural extensions: integrating a process reward model to provide verification signals where symbolic checking fails, and improving the LaTeX-to-SymPy parser to handle a broader range of mathematical expressions.

Table 10: Tree structure statistics by problem difficulty.

Difficulty	Avg Depth	Avg Branching	Prune Rate	Nodes Explored
Easy	—	—	—	—
Medium	—	—	—	—
Hard	—	—	—	—

6.5 Transfer Effect: Does MCTS-RL Training Improve Greedy Decoding?

A critical question is whether the MCTS-RL-trained model is better *only* when used with MCTS at inference time, or whether it has internalized the verification signal and produces better solutions even with greedy decoding (no MCTS).

Table 11: Accuracy (%) of MCTS-RL model with and without MCTS at inference.

Model	Greedy (no MCTS)	With DS-MCTS
SFT model	—	—
MCTS-RL model	—	—

7 Discussion

Summary of Findings. DS-MCTS with composite verification consistently improves mathematical reasoning accuracy across three base models with different pre-training regimes. The two-phase training pipeline (SFT followed by MCTS-RL) outperforms SFT alone, with the largest gains on hard, olympiad-level problems where the model produces the highest rate of symbolic contradictions. The composite verifier (symbolic + code) outperforms either component in isolation, confirming that they catch complementary error classes.

Practical Implications. The DS-MCTS framework is designed for practicality on consumer hardware. All experiments run on a single GPU (RTX 3060, 12GB VRAM) or Apple Silicon (M4, 16GB unified memory) using 4-bit quantized models with LoRA adapters. This demonstrates that meaningful improvements to mathematical reasoning do not require cluster-scale compute. The verification components—SymPy for symbolic checking and subprocess execution for code blocks—add negligible computational overhead compared to the LLM inference cost.

The finding that MCTS-RL-trained models improve greedy decoding accuracy (not just MCTS-assisted accuracy) has practical significance: after training, the model can be deployed without MCTS for latency-sensitive applications, retaining most of the accuracy improvement. MCTS can be reserved for high-stakes problems where correctness justifies additional computation.

Relationship to Formal Verification. Our symbolic verification occupies a middle ground between no verification (standard CoT/SFT) and full formal verification (Lean/Isabelle). SymPy catches a specific but important class of errors—numeric equation contradictions—without requiring the solution to be expressed in a formal proof language. This tradeoff is deliberate: formal verification provides completeness guarantees but requires autoformalization (an unsolved problem for most mathematical domains), while our approach operates directly on the natural language and code artifacts that LLMs already produce.

Limitations. Several limitations should be noted:

1. **Scale:** We evaluate only 7B-parameter models. Whether DS-MCTS provides similar gains at 70B+ scale is an open question; larger models may make fewer symbolic errors, reducing the verifier’s impact.
2. **Verifier coverage:** The symbolic verifier only catches errors in equations it can parse. Non-standard notation, implicit reasoning, and geometric arguments are unverifiable with our approach.
3. **Code isolation:** Each code block executes in an independent subprocess without shared state. TIR solutions that depend on variables computed in previous code blocks will fail the code verifier even if the code is semantically correct. This could be addressed by maintaining a persistent execution environment, at the cost of security and isolation.
4. **Compute cost:** MCTS at inference time is slower than greedy decoding by a factor proportional to $T \times b$ (iterations times branching factor). This tradeoff is acceptable for competition settings (where time limits are generous) but may be prohibitive for real-time applications.
5. **Benchmark scope:** All experiments use the OpenMathReasoning evaluation subset. Validation on additional benchmarks (MATH, GSM8K, AMC, AIME) would strengthen generalizability claims.

Future Work. Several directions extend naturally from this work:

- **Process reward model integration:** Using a learned PRM as a soft preference signal alongside the hard verification constraint could improve performance on problems where symbolic checking provides no signal.
- **Adaptive search:** Learning problem-dependent MCTS hyperparameters (depth, branching, UCT constant) based on predicted problem difficulty could improve token efficiency.
- **Cross-domain application:** The DS-MCTS framework applies to any domain where intermediate reasoning steps produce verifiable claims: physics (dimensional analysis), chemistry (conservation laws), and programming (type checking, unit tests).
- **Persistent code execution:** Maintaining a shared Python environment across code blocks within a single solution would enable the code verifier to catch semantic errors (wrong intermediate values) in addition to runtime errors.
- **Scaling laws:** Characterizing how verification benefit varies with model size would inform when DS-MCTS is most valuable.

8 Conclusion

We presented Deductive-State Monte Carlo Tree Search (DS-MCTS), a framework that integrates symbolic and code execution verification into the MCTS search process for mathematical reasoning. By pruning deductively invalid reasoning branches during tree expansion, DS-MCTS focuses computation on symbolically consistent solution paths. The two-phase training pipeline—supervised fine-tuning followed by MCTS-guided rejection sampling—produces models that generate fewer symbolic contradictions and code errors even without MCTS at inference time.

Evaluation across three 7B-parameter base models demonstrates that DS-MCTS generalizes across architectures and pre-training regimes. Comprehensive ablations confirm that composite verification (symbolic + code) outperforms either component alone, that the method provides the largest gains on hard problems where models make the most errors, and that DS-MCTS achieves superior accuracy-per-token efficiency compared to self-consistency decoding.

Our results suggest that lightweight, deterministic verification can meaningfully improve LLM reasoning without requiring heavyweight formal proof systems or expensive human annotation. The framework is practical on consumer hardware and extensible to other STEM

domains where intermediate reasoning steps produce verifiable claims.

References

- Auer, P., Cesa-Bianchi, N., and Fischer, P. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2):235–256, 2002.
- Coulom, R. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games*, 2006.
- de Moura, L. and Ullrich, S. The Lean 4 theorem prover and programming language. In *CADE*, 2021.
- DeepMind. AI achieves silver-medal standard solving International Mathematical Olympiad problems. Blog post, 2024.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. QLoRA: Efficient finetuning of quantized language models. In *NeurIPS*, 2023.
- Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- Kocsis, L. and Szepesvári, C. Bandit based Monte-Carlo planning. In *ECML*, 2006.
- Li, J., et al. NuminaMath: Winning the AI Mathematical Olympiad Progress Prize. Technical report, 2024.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *ICLR*, 2023.
- Meurer, A., et al. SymPy: Symbolic mathematics in Python. *PeerJ Computer Science*, 3:e103, 2017.
- MLX Contributors. MLX: An array framework for Apple silicon. GitHub repository, 2023.
- Moshkov, N., et al. OpenMathReasoning: Advancing mathematical reasoning with large-scale synthetic data. *arXiv preprint arXiv:2504.16891*, 2025.
- NVIDIA. OpenMathReasoning dataset. Hugging Face: [nvidia/OpenMathReasoning](https://huggingface.co/nvidia/OpenMathReasoning), 2025.
- OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Ouyang, L., et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.
- Paulson, L. C. *Isabelle: A Generic Theorem Prover*. Springer, 1994.
- Shao, Z., et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Silver, D., et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- Wang, X., et al. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.

- Wei, J., et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- Wu, Y., et al. Autoformalization with large language models. In *NeurIPS*, 2022.
- Yang, A., et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.
- Yao, S., et al. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023.
- Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. STaR: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022.